Antigravity Agent Roadmap: Flavor-Aware Healthier Recipe Substitution

Oct 30, 2026 · @Someone

1. Mission brief (read this first)

You are the coding and analysis agent for a four-week research project: build and evaluate a flavor-aware healthier recipe substitution pipeline. Project 2 from the proposal (constraint-aware recipe generation with a checker, LLM prompting, LoRA fine-tuning) is out of scope. Do not build any of it.

One line. Given a recipe, replace 1 to 3 ingredients that contribute most to a nutrient we want to reduce (calories, saturated fat, sugar or sodium) with healthier alternatives, keep the dish plausible, and find which compatibility measure best identifies good swaps.

Who you work for. Two undergraduate students new to ML, supervised by Prof. Ganesh Bagler (IIIT Delhi; RecipeDB, FlavorDB, Foodoscope). The students must understand everything you build.

Research questions

1. Which measure best identifies an ingredient that fits a recipe: co-occurrence (PMI), flavor sharing, a learned embedding, or a combination?
2. Does FlavorDB flavor-molecule information improve on co-occurrence alone?
3. Does the answer differ across cuisines? The lab's 2015 study found negative food pairing in Indian regional cuisines (more shared flavor compounds, less co-occurrence), so never assume flavor sharing means compatibility. Test it.
4. How much estimated nutrition improvement can swaps achieve while staying plausible?

New requirement from the professor. Besides the automatic hide-and-recover test, we manually label a set of ingredient pairs and compare the model's compatibility scores with those human labels. The full protocol is in Section 7; it is scheduled in Weeks 2 and 3.

How you work

- Explain in simple language. Define every ML term the first time with a tiny example, in code comments or the notebook text.

- Work in small steps. After each step, print the key outputs and a one-paragraph note of what they mean.
- Never invent numbers, column names or dataset facts. If unsure, inspect the data and say what you found.
- Always compare with a simple baseline, say when a difference is within noise, and report null or negative results.
- At the end of every week, stop at the gate (below), write a short status note with evidence, and wait for the students to confirm before starting the next week.

2. Ground rules, data and repository layout

Data sources

Source	Use	Status
RecipeDB	Recipes with ingredients parsed into name, quantity, unit, state; 29 ingredient categories; 5 dietary styles; estimated nutrition. Paper reports 118,171 recipes	Access (bulk vs API, rate limits) to be confirmed with the professor
FlavorDB	Flavor molecules per ingredient	Mapping from RecipeDB ingredients to be confirmed
Foodoscope APIs	Substitution and compatibility output as baseline B5	Only if access is granted
Food.com (earlier project)	Prototype compatibility measures only: no quantities, recipe-level nutrition only	Available now

Fallbacks. If RecipeDB access is slow, prototype the compatibility part on Food.com and keep every function source-agnostic. If nutrients per ingredient are sparse, restrict to the nutrients available and state this in the report.

Data caveats you must respect

- Food.com cannot support per-ingredient health calculations, so never use it for the health part.
- RecipeDB nutrition is estimated, so every improvement is an estimated improvement.

- Flavor molecules do not capture texture or cooking function (butter or sugar in baking). Start with function-safe swaps and document exclusions.
- "Healthier" is defined explicitly: the targeted nutrient per serving drops by at least 15%, protein stays within 20% of the original, and the dietary style is preserved. Thresholds are placeholders until the professor confirms. Keep them in one config file. This is a computational study, not medical advice.

Leakage rules (non-negotiable)

- Split by recipe into train, validation and test (for example 70/15/15) with a fixed seed. Never reuse a recipe across splits.
- Compute PMI, co-occurrence counts, embeddings, thresholds and ingredient vocabulary from training recipes only.
- Tune weights and hyperparameters on validation only. Touch the test set once, for final numbers.
- Manually labelled pairs (Section 7) must be labelled before the agent shows the students any model scores for them.

Repository layout

```
recipe-swap/
  README.md          how to run everything, versions, seeds
  config.yaml        thresholds, seeds, paths, nutrient list
  data/raw/          never edited
  data/processed/    cleaned tables, mappings, splits
  src/               parsing, grams, nutrition, measures, candidates,
scoring, eval
  notebooks/         one per week, plain-language explanations
  labels/            manual pair labels and rater sheets
  results/           tables, figures, bootstrap outputs
  reports/           weekly status notes, final report
```

Stack. Python, pandas, numpy, scikit-learn, matplotlib, LightGBM, gensim (Word2Vec) or a small PyTorch model for embeddings. Record package versions in the README.

3. Week 1: data, cleaning, mappings, coverage

Goal. A parser and mapping layer the rest of the project can trust.

1. **Data access and inspection.** Load whatever RecipeDB, FlavorDB and Food.com data exists. Report counts, columns, dtypes, missing rates, which nutrients exist per ingredient (sugar, sodium, saturated fat?), and cuisine labels. Do not assume column names; print them. Output: `reports/week1_data_inventory.md`.
2. **Literature check.** Search for prior work on ingredient substitution, food pairing and healthier swaps (start with Shirai et al. 2021 and the lab's 2015 food-pairing paper). Output: a one-page note listing overlaps and how our comparison of measures differs. Flag anything that threatens novelty.
3. **Ingredient cleaning.** Reuse the earlier cleaning rules (lowercase, singularize, strip prep words like "diced", split compounds like "salt and pepper"). Handle ambiguous words ("clove" as spice vs garlic). Output: `data/processed/ingredient_normalization.csv` (raw name, clean name, rule applied).
4. **Quantity to grams.** Parse quantity, unit and state; convert to grams with a density or unit table. Log every ingredient that cannot be converted. Output: grams column plus an unconverted-ingredients report.
5. **Nutrition and flavor mapping.** Map each clean ingredient to nutrition per 100 g and to a FlavorDB entry. Output: `ingredient_master.csv` with category, nutrition per 100 g, FlavorDB id (or blank), mapping method and confidence.
6. **Coverage report.** Share of ingredients, and of recipe weight, that have both nutrition and flavor data, overall and per cuisine and category. Decide a minimum-coverage rule for recipes to be kept.
7. **Splits.** Create the recipe-level train, validation and test split with a fixed seed and save the recipe ids. Check cuisine balance in each split.
8. **Hand-check.** Students and agent together hand-check 50 random recipes for parsing and mapping errors and record the error rate.

Gate 1. The parser works on the hand-checked sample (target: at least 90% of ingredient lines correct, to be confirmed with the students) and coverage is acceptable. If not, fix before continuing, or switch the compatibility work to Food.com and say so.

Questions to raise with the professor this week: bulk data vs API and rate limits, nutrients available per ingredient, the RecipeDB to FlavorDB mapping, the preferred definition of "healthier", and an expert who could rate swaps.

4. Week 2: nutrition, candidates, compatibility measures, hide-and-recover

Goal. Every compatibility measure exists, is leakage-free, and is tested objectively. Work splits naturally in two tracks that can run in parallel.

Track A: health side (Person 1)

1. **Nutrition per serving.** Quantity in grams times nutrition per gram, summed per recipe and divided by servings. Validate on a 200-recipe sample against RecipeDB's listed values; report the median and 90th-percentile error.
2. **High-impact ingredients.** For each recipe and target nutrient, rank ingredients by contribution. Report how concentrated contribution is (share from the top 1, 2, 3 ingredients).
3. **Candidate generation.** For each target ingredient, list candidates from the same RecipeDB category plus a small curated function-safe set (for example fats and oils), with a lower value of the targeted nutrient per 100 g and a compatible dietary style. Save the curated groups and exclusions in a reviewed file.

Track B: compatibility side (Person 2)

4. **PMI (co-occurrence).** From training recipes only. Define the recipe-level score of a candidate as the mean (and also max) PMI with the other ingredients. Handle rare pairs with a minimum count and smoothing; document the choice.
5. **Flavor sharing.** Jaccard overlap of FlavorDB molecule sets between candidate and each other ingredient. Report how many ingredients lack flavor data and how those cases are treated (never silently score as zero).
6. **Learned embeddings.** Treat each training recipe as a sentence of ingredients; train Word2Vec (or similar) and use cosine similarity. Fix the seed and report sensitivity to dimension and window size.
7. **Hide-and-recover.** In validation recipes hide one ingredient, rank candidates (all ingredients, then same-category only) with each measure and a random baseline, and report top-1, top-5, top-10 and MRR, overall and per cuisine. Keep cuisines with too few recipes out of the per-cuisine table and say so.

Track C: manual pair set (both, start now)

8. **Freeze the pair set.** Build the list of ingredient pairs to be labelled by hand, following Section 7. Freeze it before any model score is looked at.

9. **Begin independent labelling.** Each student labels the pairs alone. Finish by early Week 3.

Gate 2. Hide-and-recover runs end to end with all four methods plus a random baseline, and recomputed nutrition matches RecipeDB on the sample within the tolerance the students agree on. If nutrition does not match, find out why before building swaps on it.

5. Week 3: supervised model, baselines, swaps, human evaluation

Goal. The full swap pipeline runs, is compared with baselines, and is checked against human judgement.

1. **Supervised compatibility model (the core ML part).** Training labels come free from recipes: hide one ingredient, the true ingredient is a positive, random ingredients are negatives. Features: PMI, shared flavor molecules, embedding similarity, ingredient category, cuisine. Models: logistic regression first, then LightGBM. Evaluate on held-out recipes against a PMI-only baseline. Inspect feature importance and per-cuisine behaviour to see where flavor sharing helps or hurts.
2. **Manual-label comparison.** Compare model and single-measure scores with the students' manual pair labels, as set out in Section 7.
3. **Scoring function.** Score = a x health gain + b x similarity to original + c x compatibility with remaining ingredients. Tune weights on validation recipes only; report sensitivity to the weights.
4. **Apply swaps.** Up to 3 swaps per recipe at equal weight, recompute nutrition, record before and after.
5. **Baselines.** B1 random same-category swap; B2 healthiest-only swap; B3 co-occurrence only; B4 flavor sharing only; B5 Foodoscope substitution if access exists.
6. **Health and preservation outcomes.** Percent reduction in the targeted nutrient, share of swaps meeting the health criterion, protein side effects, same-category rate, flavor-profile similarity before and after.
7. **Human study of swaps.** Sample 40 to 50 swaps across cuisines and methods (blind to which method produced them, shuffled). Both students rate plausibility and taste preservation from 1 to 5 independently. Compute mean per method and Cohen's kappa. Ask the professor for an expert to rate a subset.
8. **Cuisine-wise analysis.** Repeat key comparisons per cuisine with enough recipes.

Gate 3. Human ratings and manual pair labels are complete, baselines have run on the validation set, and the swap pipeline produces explained outputs for a sample of recipes. Test set still untouched.

6. Week 4: final test run, report, demo

Goal. Final numbers, honest write-up, and a demo the students can present.

1. **Freeze.** Freeze weights, models and thresholds. Run everything once on the test set with the fixed seed. No further tuning.
2. **Statistics.** Bootstrap 95% confidence intervals (resampling recipes, at least 1,000 resamples) for every comparison: hide-and-recover metrics, health outcomes, human ratings and manual-label agreement. State when a difference is within noise.
3. **Cuisine-wise tables and null results.** Report where flavor information helps, where it hurts, and where nothing differs. Include failure cases and unrealistic swaps.
4. **Examples.** At least 10 transformed recipes with before and after ingredients, nutrition, compatibility scores and a plain explanation, including 2 or 3 bad swaps.
5. **Report.** Sections: introduction and related work (honest novelty statement), data and mapping with coverage, methods, manual labelling study, results, limitations, conclusion. Make clear which parts are ML and which are data engineering.
6. **Demo.** A notebook or small web app: choose a recipe and target nutrient, see ranked swaps with scores and nutrition before and after.
7. **Slides.** 10 to 12 slides following the report.
8. **Reproducibility.** README with exact run order, seeds and package versions; a fresh-environment test run of the main notebooks.

Gate 4 (final). All deliverables in Section 8 exist and the students can explain each result without the agent.

7. Manual ingredient-pair labelling and model comparison

Why. Hide-and-recover only tells us whether a measure agrees with what recipes contain. It cannot tell us whether a pair works to a human cook. Labelling pairs by hand gives an independent check on every model, and is the study the professor asked for.

Pair set (target about 150 pairs, to be adjusted to student time)

- **Type A, compatibility pairs (about 100).** Two ingredients, asked: would a cook plausibly use these two together in the same dish?

- **Type B, substitution pairs (about 50).** An original ingredient, a candidate replacement and the recipe or cuisine context, asked: could the candidate replace the original here without ruining the dish?
- **Sampling.** Stratified, not random only. Fill four cells of PMI (high or low) by flavor sharing (high or low), so we see where the measures disagree; spread across cuisines and categories; add about 10 obvious good pairs and 10 obvious bad pairs as sanity anchors. Use common ingredients so the students can judge them.
- **Frozen before scoring.** The pair list is fixed before anyone looks at model scores on it.

Label scale. 1 = does not work, 2 = possible or unusual, 3 = works well, plus "?" when the rater does not know the ingredient. Add a free-text note column for reasons.

Labelling sheet columns

Column	Content
pair_id	Shuffled id, no method name
type	A or B
ingredient_1, ingredient_2	Clean names (for B: original and candidate)
context	Cuisine, and recipe title for type B
rater1_label, rater2_label	Filled independently, hidden from each other
notes	Optional reason
consensus_label	Filled only after both are done

Model scores are kept in a separate file the raters never see until all labels are locked.

Procedure

1. The agent generates the blinded sheet and an answer-key file with scores and strata.
2. Each student labels alone, without searching for model output.
3. Agent computes weighted Cohen's kappa (ordinal) between the two raters. If kappa is low, tighten the written guidelines and discuss disagreements.
4. Disagreements are resolved by discussion into a consensus label; "?" pairs are dropped and counted.
5. If the professor can suggest an expert, they rate a subset of 30 to 40 pairs, and agreement with the students is reported.

Comparison with the model

- Score every pair with PMI, flavor sharing (Jaccard), embedding cosine, the supervised model, and a random baseline, all computed from training recipes only.
- Metrics: Spearman correlation between score and label; AUROC for separating label 3 from label 1; precision at top 20% of scores; each with bootstrap 95% confidence intervals over pairs.
- Breakdowns: by type A and B, by cuisine, and by the four PMI by flavor cells. Specifically report which measure agrees with the humans when PMI and flavor sharing disagree.
- Compare against the hide-and-recover ranking: do the measures rank in the same order on both tests? If not, say so and discuss.
- Report the model's confident mistakes (high score, label 1 and the reverse) as examples.

Limits to state in the report. Two students are not expert cooks, so labels reflect their cultural familiarity; the set is small, so confidence intervals will be wide; a pair that is rare in recipes may still be a good pair. Do not oversell results.

8. Baselines, risks and deliverables checklist

Risks and what the agent should do

Risk	Response
RecipeDB or FlavorDB access delayed	Prototype compatibility on Food.com; keep code source-agnostic; tell the students immediately
Nutrients per ingredient sparse	Restrict to the nutrients available and state it
Flavor data does not help	Report it as a finding; co-occurrence stays the main measure
Swaps look unrealistic	Function-safe categories only; use the human study to filter
Mapping errors	Hand-check samples; log confidence of each mapping
Results within noise	Say so with the confidence interval; do not rank methods that overlap

Risk	Response
Overlap with existing work	Literature note in Week 1; reposition honestly, no novelty claims beyond a controlled per-cuisine comparison

Stop and ask the students when: a threshold or definition must be chosen (for example "healthier"), a dataset fact is uncertain, a result looks too good (check for leakage first), or a gate fails.

Deliverables checklist

- [ ] Reproducible code and notebooks, one per week, with plain-language explanations
- [ ] Ingredient normalization table and nutrition/flavor mapping with coverage report
- [ ] Hide-and-recover table with confidence intervals, overall and per cuisine
- [ ] Manual pair labels, rater agreement, and model-versus-human comparison table
- [ ] Baseline comparison (B1 to B4, B5 if available) with health and preservation outcomes
- [ ] Human rating of swaps with Cohen's kappa
- [ ] At least 10 transformed recipes with explanations
- [ ] Final report, slides and short demo
- [ ] README with seeds, versions and run order